



Streams & Lambda Problem Bank (Java)

Legend

★ = Core Placement Problem (Mandatory)

These problems should be completed by every student before moving to Multithreading.

Section 1: Lambda Expression Basics

★ Lambda Expression to Add Two Numbers

★ Lambda Expression to Find Square of a Number

Lambda Expression to Check Even or Odd

★ Lambda Expression with Multiple Parameters

Lambda Expression with No Parameters

Lambda Expression vs Anonymous Inner Class Comparison

Section 2: Functional Interfaces (Java-Specific)

★ Custom Functional Interface using @FunctionalInterface

★ Predicate Interface — Testing a Condition

★ Function Interface — Transforming a Value

★ Consumer Interface — Performing an Action

Supplier Interface — Generating a Value

BiFunction Interface — Combining Two Values

UnaryOperator and BinaryOperator Usage

Section 3: Stream Creation and Basics

★ Create a Stream from a Collection using stream()

★ Create a Stream using Stream.of()

Create an Infinite Stream using Stream.iterate() with limit()

Create a Stream of Random Numbers using Stream.generate()

★ Convert an Array to a Stream using Arrays.stream()

Section 4: Intermediate Stream Operations

★ Transform Elements using map()

★ Filter Elements using filter()

★ Remove Duplicates using distinct()

★ Sort Elements using sorted()

Limit the Number of Elements using limit()



Skip Elements using skip()

★ **Flatten Nested Collections using flatMap()**

Peek at Elements During Processing using peek()

Section 5: Terminal Stream Operations

★ **Collect Results using collect()**

★ **Reduce Elements using reduce()**

★ **Perform an Action on Each Element using forEach()**

Count Elements using count()

★ **Check Conditions using anyMatch()/allMatch()/noneMatch()**

Find the First Matching Element using findFirst()

Find Minimum/Maximum using min()/max()

Section 6: Collectors

★ **Collect Stream Results into a List using Collectors.toList()**

Collect Stream Results into a Set using Collectors.toSet()

★ **Collect Stream Results into a Map using Collectors.toMap()**

★ **Join Strings using Collectors.joining()**

★ **Group Elements using Collectors.groupingBy()**

Partition Elements using Collectors.partitioningBy()

Count Elements per Group using groupingBy() and counting()

Section 7: Method References (Java-Specific)

★ **Static Method Reference**

★ **Instance Method Reference of a Particular Object**

Instance Method Reference of an Arbitrary Object of a Type

★ **Constructor Reference**

Section 8: Combining map, filter, and reduce Pipelines

★ **Square Only Even Numbers using map() and filter()**

★ **Sum of Squares of Even Numbers using map(), filter(), reduce()**

Filter then Map a List of Custom Objects (e.g. Student Records)

★ **Find Sum of Lengths of Words Longer than 3 Letters**

Sort and Collect a Filtered Stream

Chain map, filter, and reduce to Process Sales Data

Section 9: Real World Streams Applications



Problem Statements

- ★ **Apply Discount to a List of Prices using map()**
- ★ **Filter Eligible Students Based on Marks using filter()**
 - Calculate Total Revenue using reduce()
- ★ **Extract a Specific Field from a List of Objects using map()**
 - Validate a List of Emails using filter() and Predicate
- ★ **Functional Pipeline for Employee Bonus Calculation**
- ★ **Group Employees by Department using Collectors.groupingBy()**

Streams & Lambda (Java)